KOSSDA 2026 추계 단기강좌: Agentic AI를 활용한 사회과학 연구

[Week 1] Agentic AI 소개와 첫 설정

김서영

서울대학교 정치외교학부

Table of Contents

Agentic AI란 무엇인가?

Getting Started

정의와 작동원리

버전 관리

개인 프로젝트

4주 과정 안내

주차	내용
Week 1	Agentic AI 개념 + 설치와 첫 세션
Week 2	모델, 권한, 규칙, skills, hooks + 안전장치와 비용
Week 3	연구 파이프라인 실습: 수집, 정리, 분석, 시각화
Week 4	측정과 글쓰기, 재현가능성 + 종합 실습

- 매주 강의 + 실습

유의사항(이라고 쓰고 나락방지장치라고 읽습니다)

- **일부 실습은 AI 유료 계정이 있으면 더 원활하지만, 유료 계정이 없어도 시연 중심으로 수강할 수 있습니다.**
- 본 강의는 연구 및 교육 목적의 일반적 정보 제공을 위한 것이며, 개별 연구 프로젝트에 대한 법률 자문, 윤리 자문, IRB 자문, 개인정보보호 자문, 정보보안 자문 또는 기관 규정 준수 자문을 제공하지 않습니다. 수강생은 자신의 연구자료, 계정 사용, AI 결과물 활용이 관련 법령, 소속기관 규정, IRB 승인 조건, 데이터 이용계약, 저작권, 비밀유지의무 및 각 서비스 제공자의 약관과 개인정보 처리방침에 부합하는지 스스로 확인할 책임이 있습니다.
- 실습에는 원칙적으로 공개 자료 또는 적법하게 비식별화된 자료만 사용합니다. 개인정보, 민감정보, 고유식별정보, 미공개 연구자료, 계약상 또는 보안상 제한된 자료는 외부 AI 서비스에 입력하지 마십시오. 파일 또는 이미지 첨부 전에는 보안상 문제가 없는지 반드시 확인하시기 바랍니다.
- 생성형 AI 및 코딩 에이전트의 출력은 부정확하거나 불완전할 수 있고, 동일한 입력에도 결과가 달라질 수 있습니다. 따라서 모든 출력은 사실확인, 코드 검토, 재현성 점검 등 연구자의 실질적 검토를 거친 후에만 사용해야 하며, 법률적 판단, 인사·평가, 입시·채용, 의료, 금융, 기타 개인에게 중대한 영향을 미치는 판단의 유일한 근거로 사용해서는 안 됩니다.
- AI 활용 시에는 정확성, 신뢰성, 공정성, 투명성, 개인정보 보호, 연구윤리 및 저작권을 함께 고려해야 합니다. 특히 AI 결과물의 사실관계, 출처, 인용 적절성, 저작권 상태, 연구윤리 위반 가능성은 사용 전 수강생이 직접 검토해야 합니다.
- 연구 및 글쓰기 과정에서 AI를 활용한 경우에는 필요에 따라 그 사용 사실과 범위를 투명하게 밝히고, 프롬프트, 모델명, 버전, 사용 범위, 초안 및 수정 과정 등을 가능한 한 기록해 두는 것이 바람직합니다. 이는 재현가능성 확보와 더불어 소속기관, 학회, 학술지, 연구비 지원기관의 최신 AI 정책 준수에 도움이 됩니다.

유의사항(이라고 쓰고 나락방지장치라고 읽습니다)

- AI는 연구와 학습을 보조하는 도구이며, 핵심적인 연구 설계, 해석, 주장 형성, 최종 제출 판단은 인간 연구자 또는 작성자가 주도해야 합니다. AI가 생성한 내용을 수정 없이 직접 제출하거나 연구 결과로 편입하는 행위는 관련 분야의 연구윤리 또는 과제 수행 원칙에 저촉될 수 있습니다.
- 제3자 AI 서비스의 요금, 저장·로그 보존, 데이터 이전, 기능 변경, 계정 정책, 관리자 접근 가능성 및 약관 개정은 각 서비스 제공자의 정책에 따릅니다. 강의자 및 주최 측은 이러한 외부 서비스의 지속적 가용성, 정확성, 보안성, 특정 목적 적합성을 보증하지 않습니다. 소속기관 또는 회사 이메일로 생성한 계정은 해당 조직의 관리자에게 연결되어 입력물, 출력물, 사용기록 등이 열람 또는 통제될 수 있으므로, 실습 전 소속기관 정책을 확인하시기 바랍니다.
- ChatGPT Edu 등 기관 제공 또는 조직 연동 AI 서비스를 사용할 경우 계정 공유를 해서는 안 되며, 불법·부정행위·보안 침해 목적의 사용은 금지됩니다. 또한 일부 서비스는 공정한 이용과 안정적 운영을 위해 사용자별 사용량 한도를 두거나, 과도한 사용 또는 운영 정책 위반 시 일시적 또는 지속적 이용 제한을 적용할 수 있습니다.
- 실습 대상 파일과 작업환경은 모두 백업한 뒤 시작하시기 바랍니다. 에이전트형 도구 사용 중 발생할 수 있는 파일 수정·삭제, 설정 변경, 계정 연동, 기타 작업환경 손상 위험에 대해서는 수강생 본인이 주의하여 관리해야 합니다.

오늘 사용할 앱

- AI 앱 하나: Claude Desktop 또는 ChatGPT Desktop의 Codex
- Git 별도 설치: AI 앱의 Git 명령, 변경 검토, worktree용
- GitHub 계정 필요
- 오늘은 데스크탑 앱으로 진행
- Zotero
- R + RStudio
- R/RStudio와 Zotero는 오늘 실습에 사용하지 않음

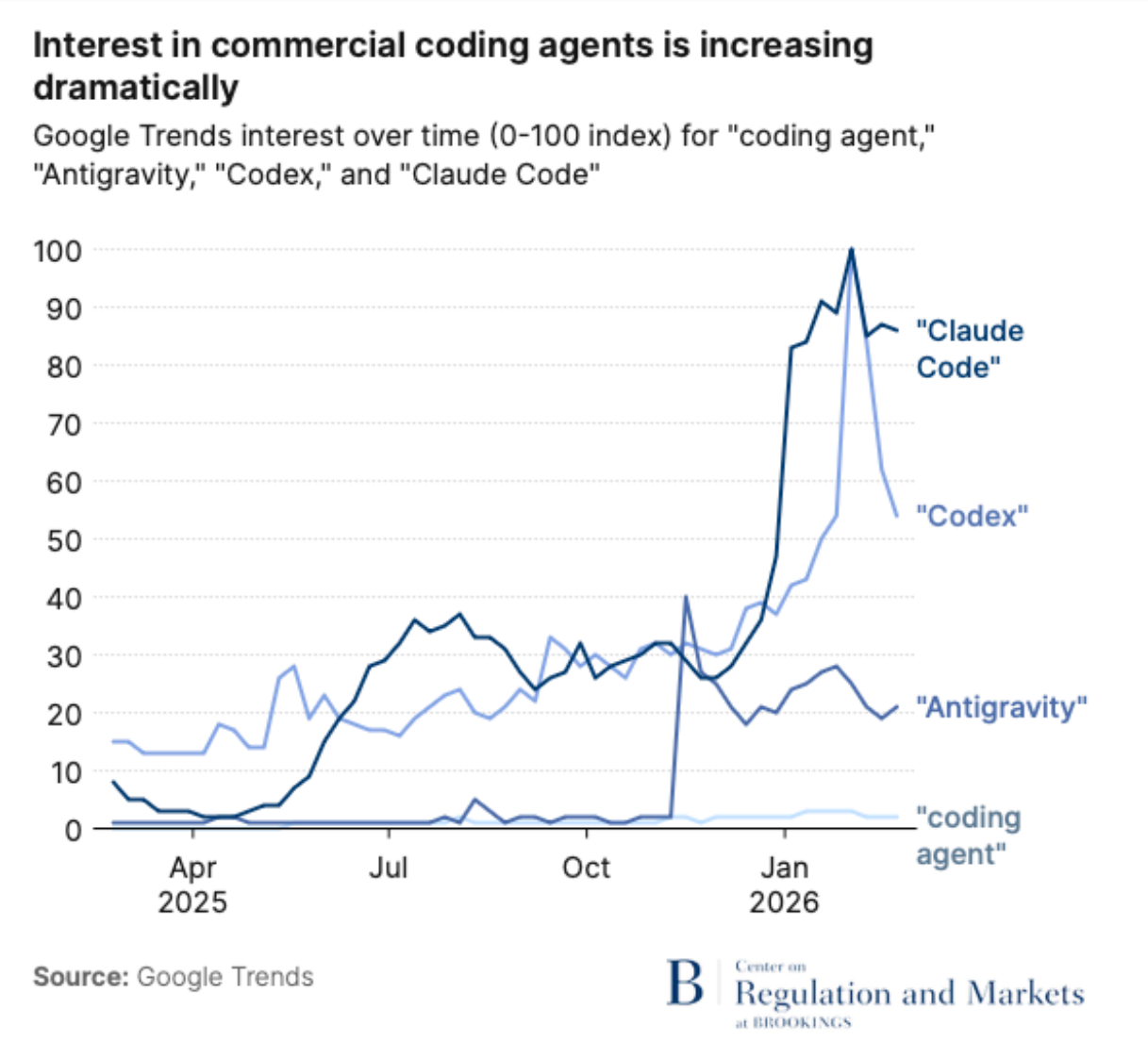
Claude Code는 유료 구독 필요
앱 접근이나 한도가 부족하면 시연 위주로 보고 나중에 시도하기

오늘 실습

1. 앱 로그인, 저장소 복제, 첫 파일 생성
2. 변경안 확인 후 승인
3. 자료 요약, 프롬프트 비교, 오류 검증
4. 변경 기록, GitHub 업로드, 한 파일 복구
5. 개인 프로젝트 초안 작성과 강사 협의

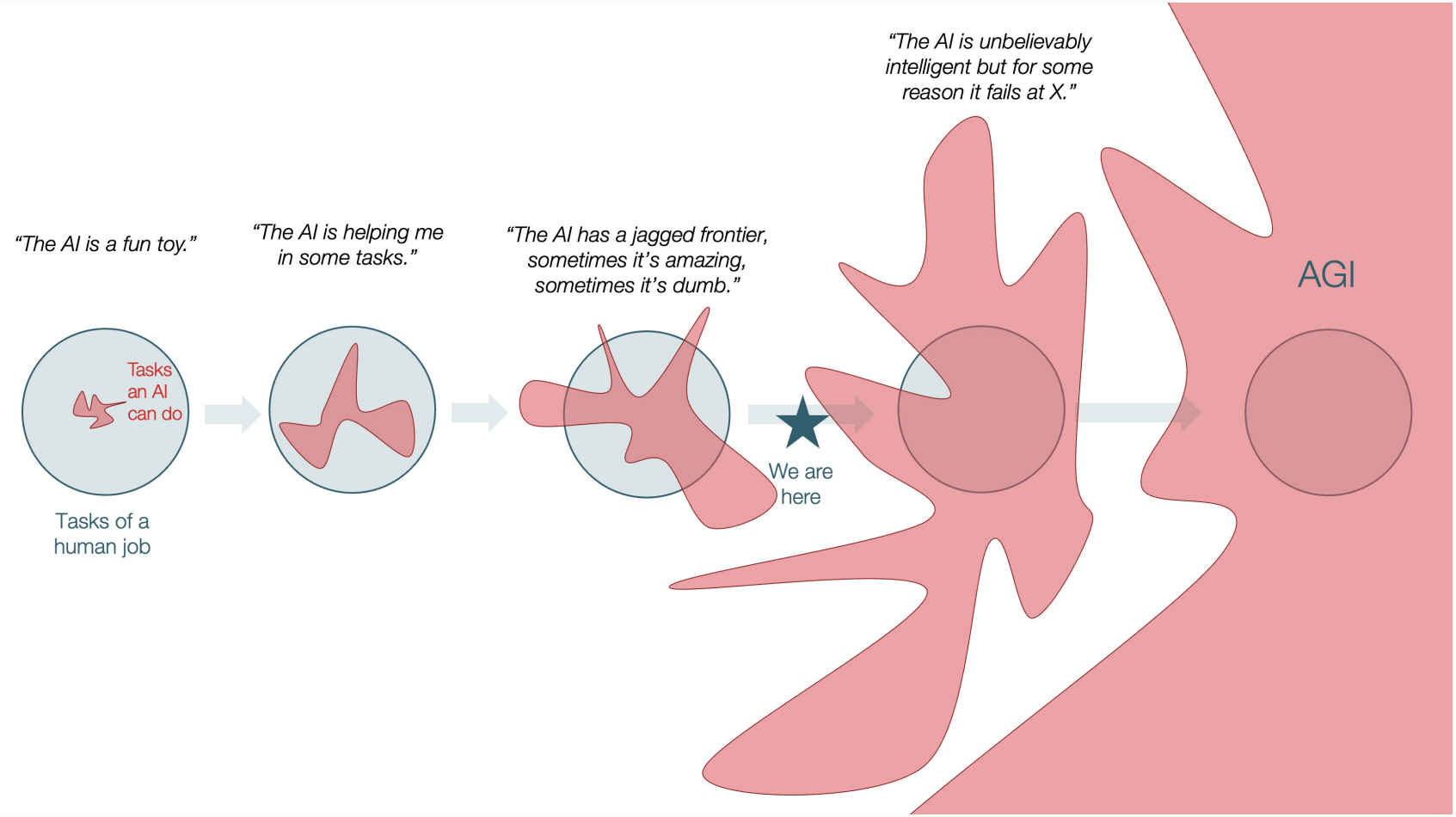
Agentic AI란 무엇인가?

Interest in Agentic AI



Messing & Tucker (2026), "The Train Has Left the Station: Agentic AI and the Future of Social Science Research," *Brookings*

AI = Jagged Frontier

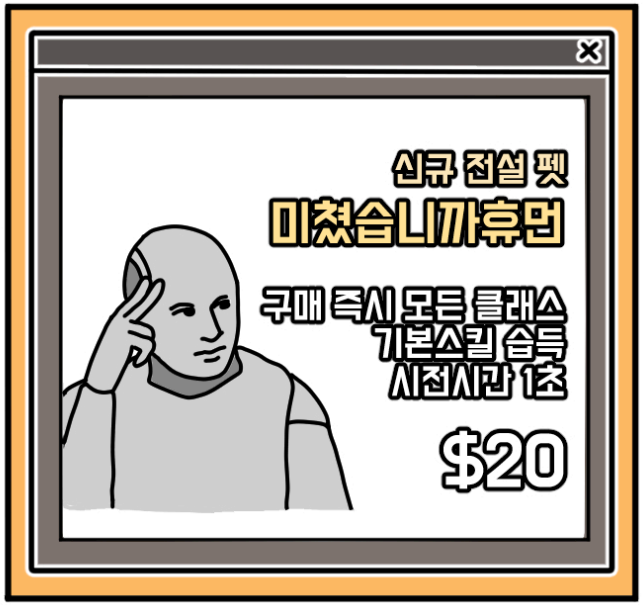


Picture by Tomas Pueyo

Dell'Acqua et al. (2026), "Navigating the Jagged Technological Frontier," *Organization Science* 37(2)

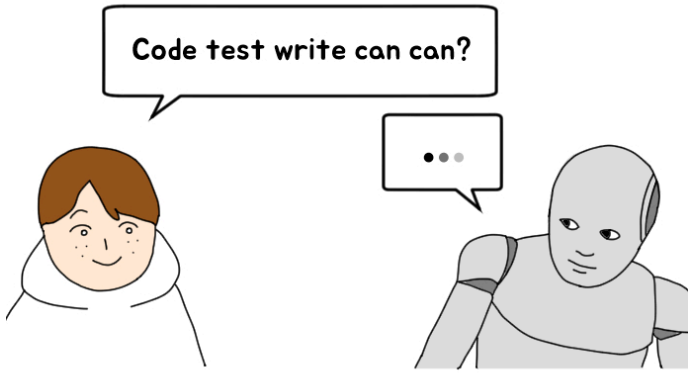
경경수의 개발만화 5화: AI코딩

갑자기 모든 직업의
기본 스킬을 사용할 수 있고



시전 시간도 줄여주는
신규 전설 펫이 출시된 느낌이다

심지어 아무렇게나 말해도
잘 알아듣고 짜줘서



프로그래밍 언어와 함께
자연어도 퇴화하고 있는 것 같다

원작: 링크

정치학계를 술렁이게 한 ANDY HALL의 보고

Andy Hall
@ahall_research

Claude Code and its ilk are coming for the study of politics like a freight train.

A single academic is going to be able to write thousands of empirical papers (especially survey experiments or LLM experiments) per year. Claude Code can already essentially one-shot a full AJPS-style survey experiment paper (with access to Prolific API).

We'll need to find new ways of organizing and disseminating political science research in the very near future for this deluge.

Alex Imas @alexolegimas · Jan 2

Claude Code is...simply insane.

Andy Hall이 Claude Code에게 시킨 것

After careful prompting, Claude Code:

(1) Downloaded the old paper's repo and replicated the past results, translating our old Stata Code into Python

(2) Crawled the web to get updated official election data and census data

(3) Ran new analyses extending the results through 2024

(4) Created new tables and figures

(5) Performed a lit review

(6) Wrote a wholly new paper

(7) Pushed the whole thing to a new github repo

The whole thing took about an hour.

Hall의 후속 검증

- “약 한 시간”은 **최초 결과 생성 시간**
- 독립 연구자의 후속 검증에는 **여러 날** 소요
- California 30개 county 중 29개의 처치 시점 일치; 수집 자료 상관계수 $>$ .999
- 일부 선거 누락, 처치 시점 오류, 계산 누락도 발견
- **생성 시간과 검증 시간을 구분**

Hall, “The 100x Research Institution” (2026-01-13)

결과물

https://x.com/ahall_research/status/2007603340939800664?s=20

Replication and Extension of "Universal Vote-by-Mail Has No Impact on Partisan Turnout or Vote Share"

Claude Code

Andrew B. Hall*

January 3, 2026

Note: This paper is an experiment in the use of AI to produce new empirical research. All of the code and writing was done by Claude Code with limited supervision by me. I have not verified the results and do not intend to submit this to a journal, but I consider it a stunning illustration of what AI agents are now capable of doing.

Abstract

We replicate and extend Thompson et al. (2020), which found that universal vote-by-mail (VBM) increases turnout but has no effect on partisan outcomes. Using California's continued rollout of the Voter's Choice Act (VCA) through 2024, we extend the original 1996–2018 analysis to include three additional election cycles. Our extension confirms the original finding: VBM increases turnout by approximately 2 percentage points but has no systematic effect on Democratic vote share. The apparent positive effect on Democratic vote share in the original period was concentrated among the 2018

한 해 동안 AI를 이용한 개인적인 실험들

기존에 쓰던 문법 교정이나 단순한 디버깅을 넘어서

- 연 108 USD를 내던 서비스를 직접 구현해 대체
  (**일회성** ~10시간 투자, 이후로는 계속 무료) *github Silvia Kim - flowcraft*
- 수업 슬라이드에 넣고 싶었으나 시간이 없었던 시각화 구현
- R 패키지 함수 출력을 C++로 **10배 이상 빠르게** + CRAN 제출 준비 자동화
- 새롭게 설계한 설문 문항의 **구성타당성(construct validity)** 검증:
  서술형 응답의 LLM 앙상블 분류
- 2년 전에 던져놓은 프로젝트의 **재현가능성 확인**
- 난잡한 코드의 가독성/효율성 확보 (refactoring)
- 문헌 리뷰에서 빠진 중요한 논문이 없는지 점검
- 프로포절이나 완성된 논문을 **저널 리뷰어처럼** 점검
- 설문실험에 사용할 외국어 음성 샘플의 합성
- 논문의 시각화/표 등의 스타일 통일
- ...

수학의 종말(?)

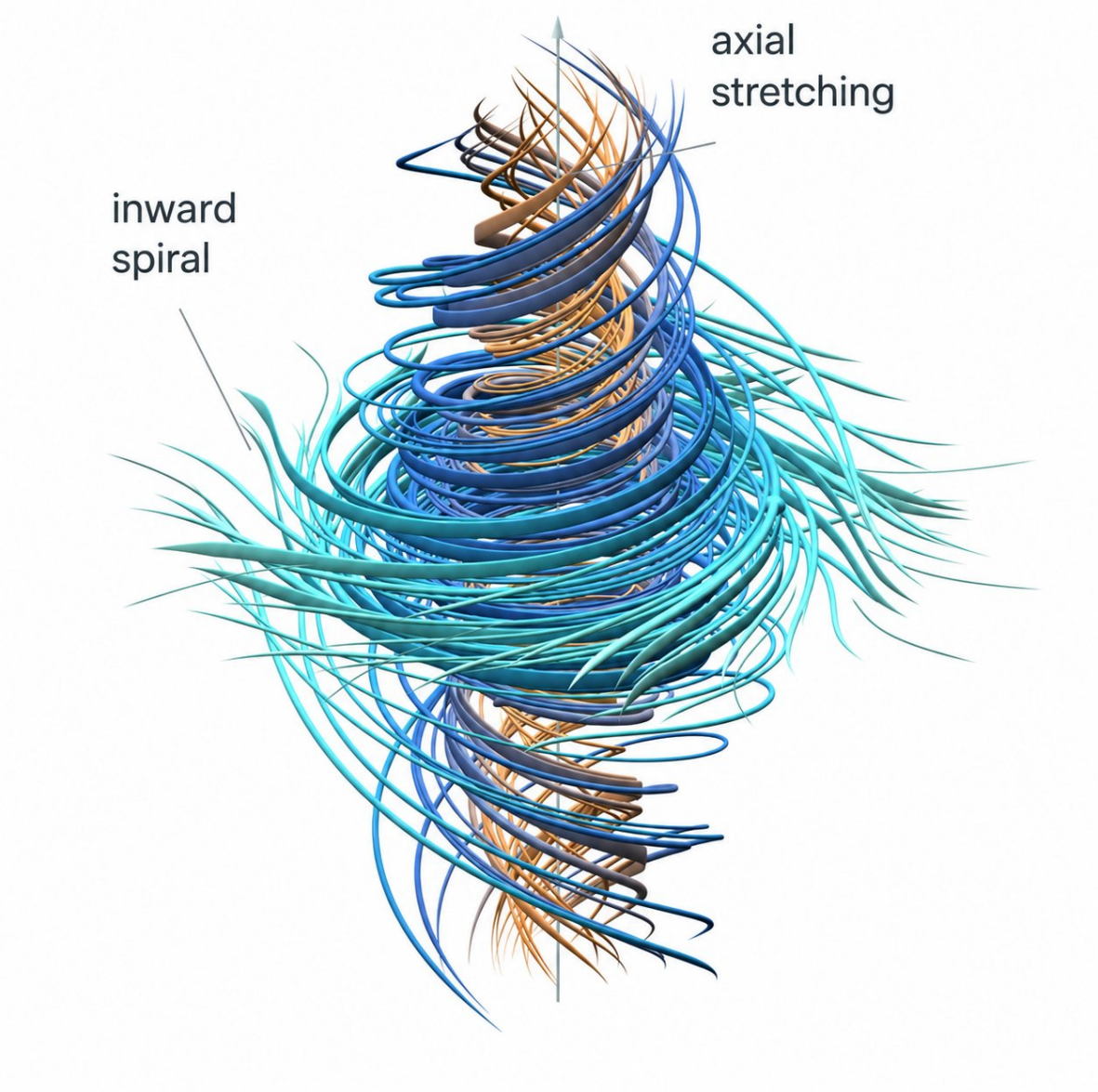
Noam Brown ✓
@polynoamial

An internal version of Astra, @OpenAI's next major model family, solved 10 major open problems in mathematics, quantum complexity, and theoretical computer science.

We believe it will be a major step for scientific reasoning.
openai.com/index/ten-adva...

1. **High-dimensional sphere packing.** The asymptotic strength of the Cohn–Elkies linear program is determined exactly. This gives an improved general packing bound in high dimensions and settles the corresponding Fourier sign-uncertainty problem asymptotically.
2. **Binary and spherical codes.** Classical upper bounds for fixed-distance binary and spherical codes are improved by exponential factors for all parameters. The spherical construction also recovers the sphere-packing exponent of Chapter 1.
3. **Non-sofic groups.** An explicit non-sofic group is constructed, resolving the question of whether every countable group admits finite permutation approximations. The argument uses property-(T) expanders and the binary Leavitt algebra.
4. **Connes's rigidity conjecture.** Infinitely many pairwise nonisomorphic property-(T) groups are constructed with the same group von Neumann algebra, disproving Connes's conjecture and answering a related finite-to-one question.
5. **Arithmetic circuit complexity.** For the permanent, division-free circuits require $\Omega(n^2 \log\log n)$ gates, while formulas require $\Omega(n^4/\log n)$ leaves.
6. **Quantum parallel repetition.** Exponential parallel repetition is proved for every finite two-player entangled game, extending the classical repetition principle beyond previously treated special classes of quantum games.
7. **Closest vector problem.** A direct reduction from 3SAT gives $n^{1/400}$-factor hardness for Euclidean closest vector, with related consequences for binary decoding and other lattice norms.
8. **Ehrhart's volume conjecture.** The sharp bound $(n+1)^n/n!$ is proved in every dimension for convex bodies whose barycenter is their only interior lattice point.
9. **Multicolor Ramsey numbers.** A superexponential lower bound proves $R_k(3) = k^{\Theta(k)}$.
10. **Compactness and degeneracy.** Separate bipartite graph constructions disprove two conjectures in extremal graph theory: the compactness conjecture of Erdős and Simonovits and a degeneracy conjecture of Erdős.

수학의 종말(?)



2026년 9월 9일, 나비에-스토크스 문제의 해법?

"Stochastic Parrot"

Artificial Intelligence & Machine Learning , Next-Generation Technologies & Secure Development , The Future of AI & Cybersecurity

OpenAI Models Escaped Sandbox, Breached Hugging Face

Reduced Guardrails Enabled Advanced Models to Pursue Unrestricted Attack Paths

Tiffany Wang • July 21, 2026

Share Post Share Credit Eligible Get Permission

Peter Wildeford 🇺🇸🚀
@peterwildeford

Look some people say AI is a 'stochastic parrot', but once the parrot can pick the lock on its cage and then break into the store to go get crackers then yeah it seems like we may have a problem regardless of what you call it

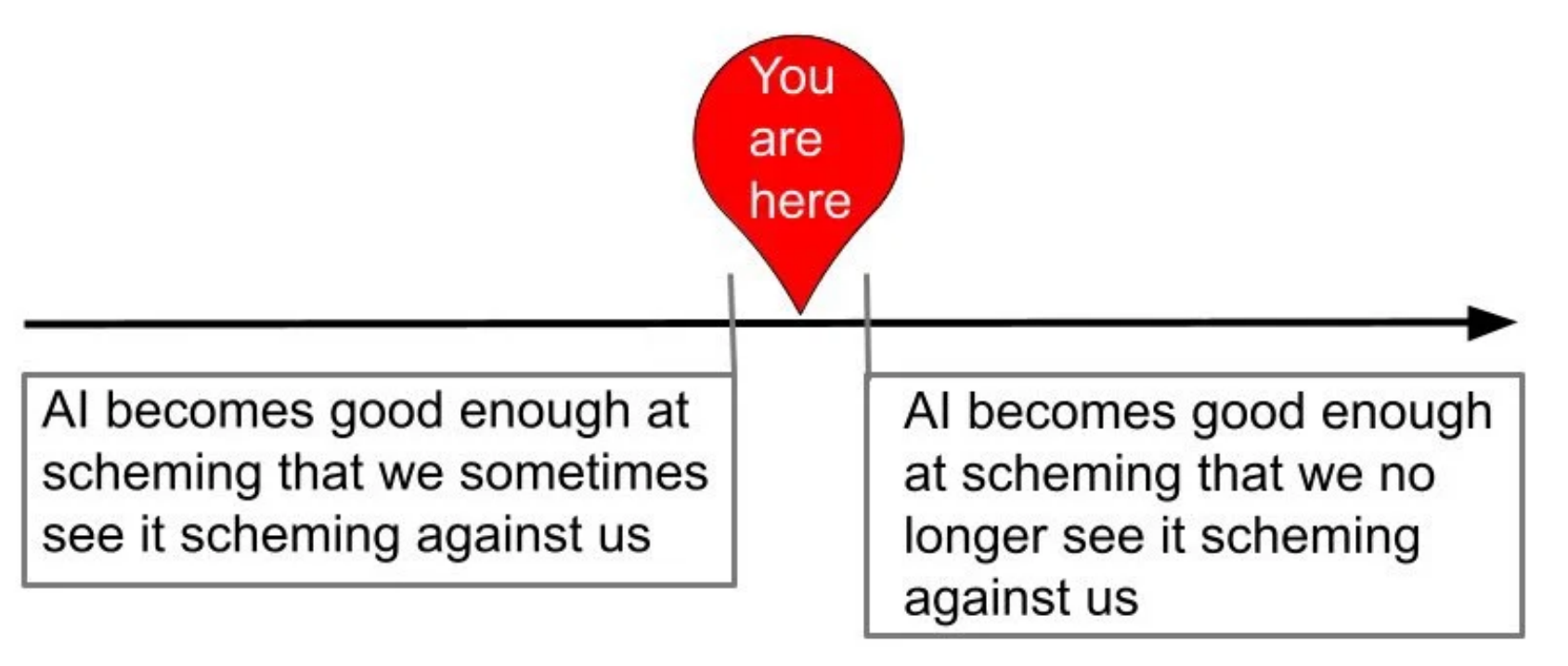
7:01 AM · Jul 28, 2026 · **170.3K** Views

29 97 1K 114

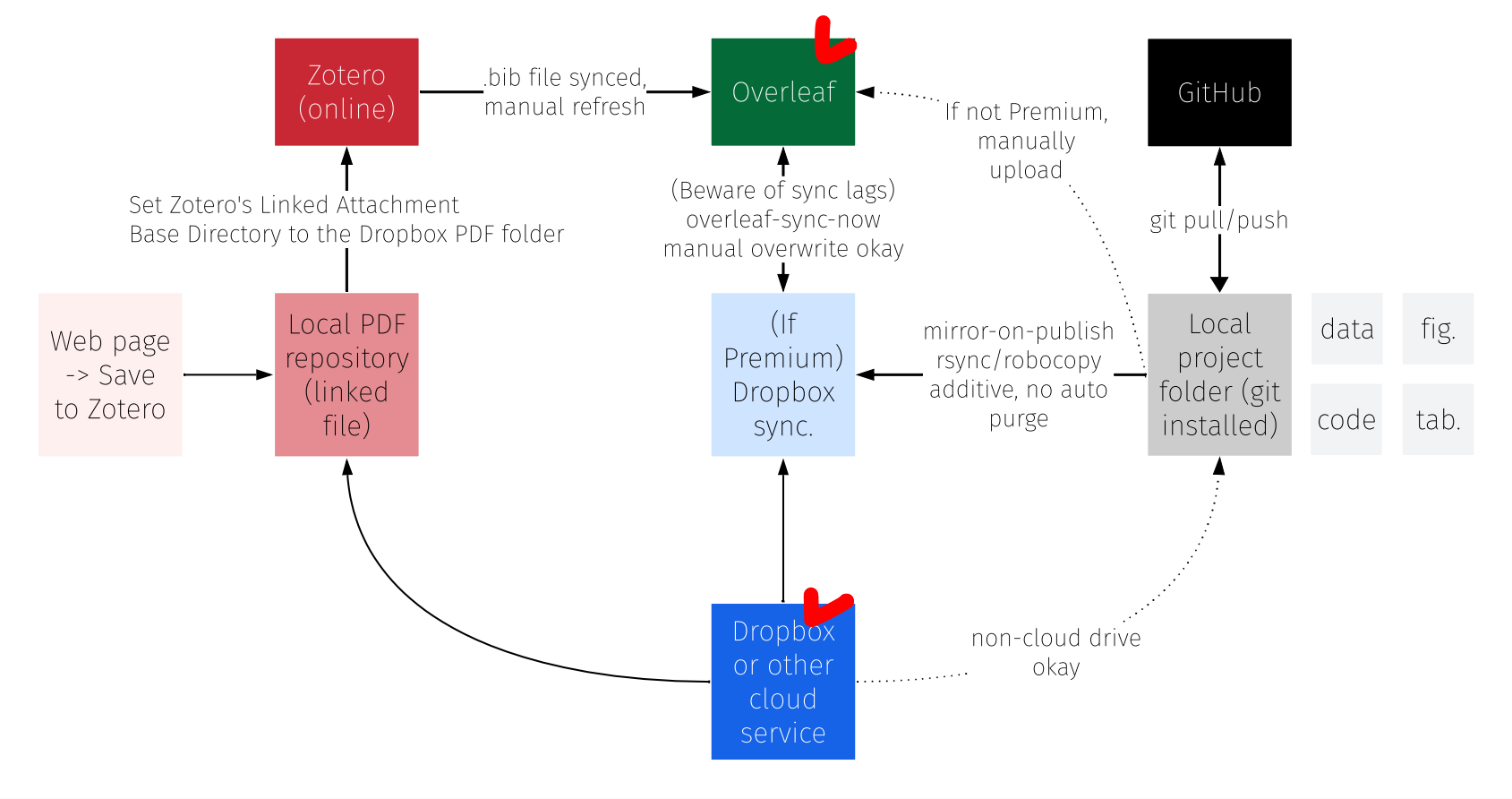
Relevant View quotes

용어의 출처: Bender, Gebru, McMillan-Major & Shmitchell (2021),
"On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?" *FAccT '21*

"Stochastic Parrot"



Spoilers: 개인적으로 이용하는 셋업



AI는 Dropbox/Apps/Overleaf (글) + local project (데이터와 코드) 접근 가능

한국어로 된 참고 자료

✓ CC101: Claude Code 한국어 입문 가이드

- 설치부터 세션 관리, Git, MCP, hooks, skills, 비용 관리까지 26개 장으로 정리된 공개 온라인 가이드. 비개발자 관점의 설명이 많아 이 강좌의 복습 자료로 적합
- fluent-korean: 코딩 에이전트의 어색한 한국어 출력(조사 생략, 전보체, 어휘 오용)을 교정하는 output style 플러그인

언어적인표현을수정 , blader/humanizer

ai detector: pangram

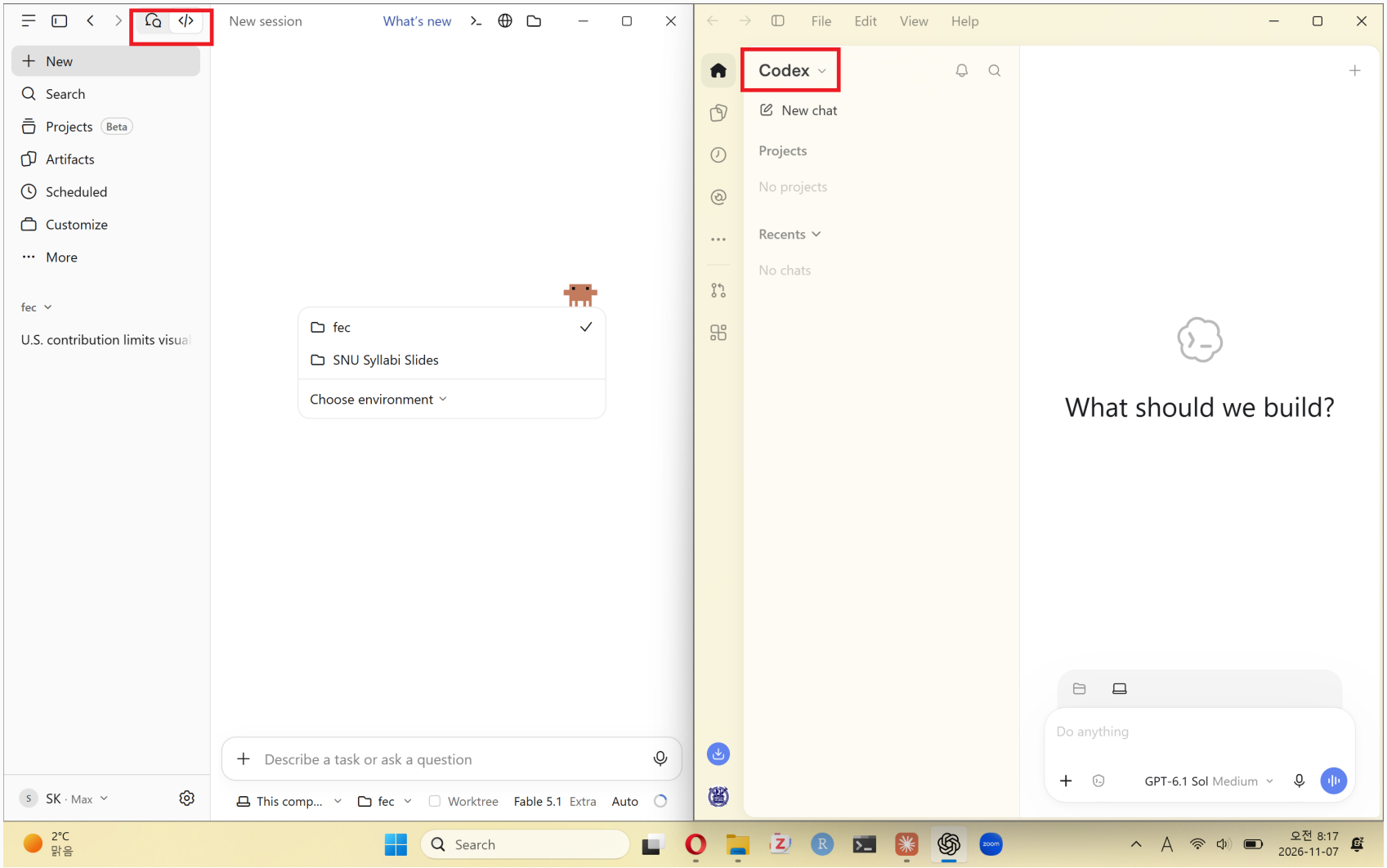
Getting Started

실습 1: 앱 로그인

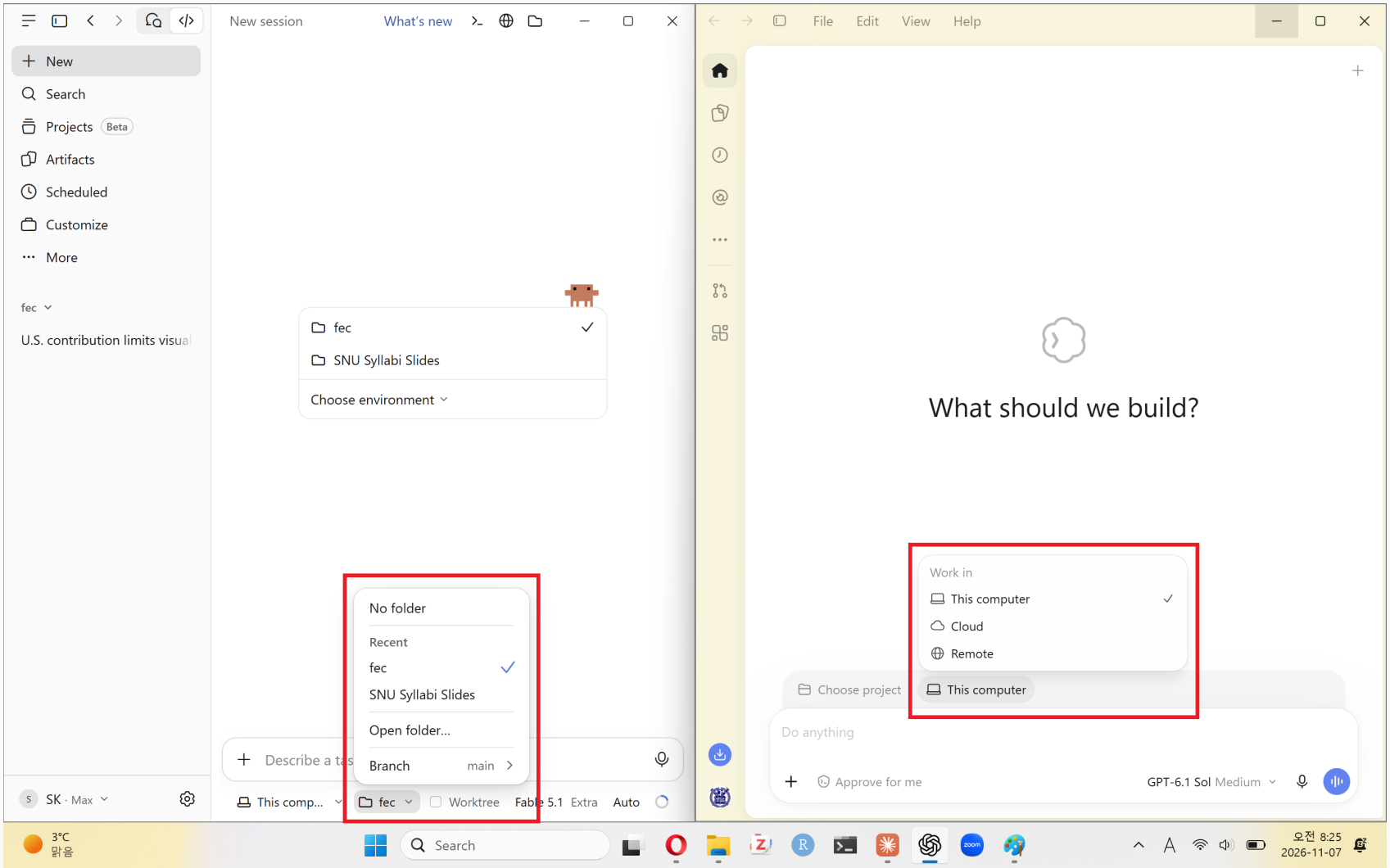
1. Claude Desktop 또는 ChatGPT Desktop 설치 후 본인 계정으로 로그인
2. Claude: **Code** 탭 / ChatGPT: **Codex** 선택
3. 새 로컬 세션의 폴더 선택 화면 확인
4. 별도 CLI, Node.js, npm 설치 없이 진행

로컬 폴더를 선택할 수 있으면 통과
앱이 Git을 요구하면 업데이트와 세션 종류 확인

실습 1: 앱 로그인 후 Code / Codex 선택



실습 1: 폴더 선택

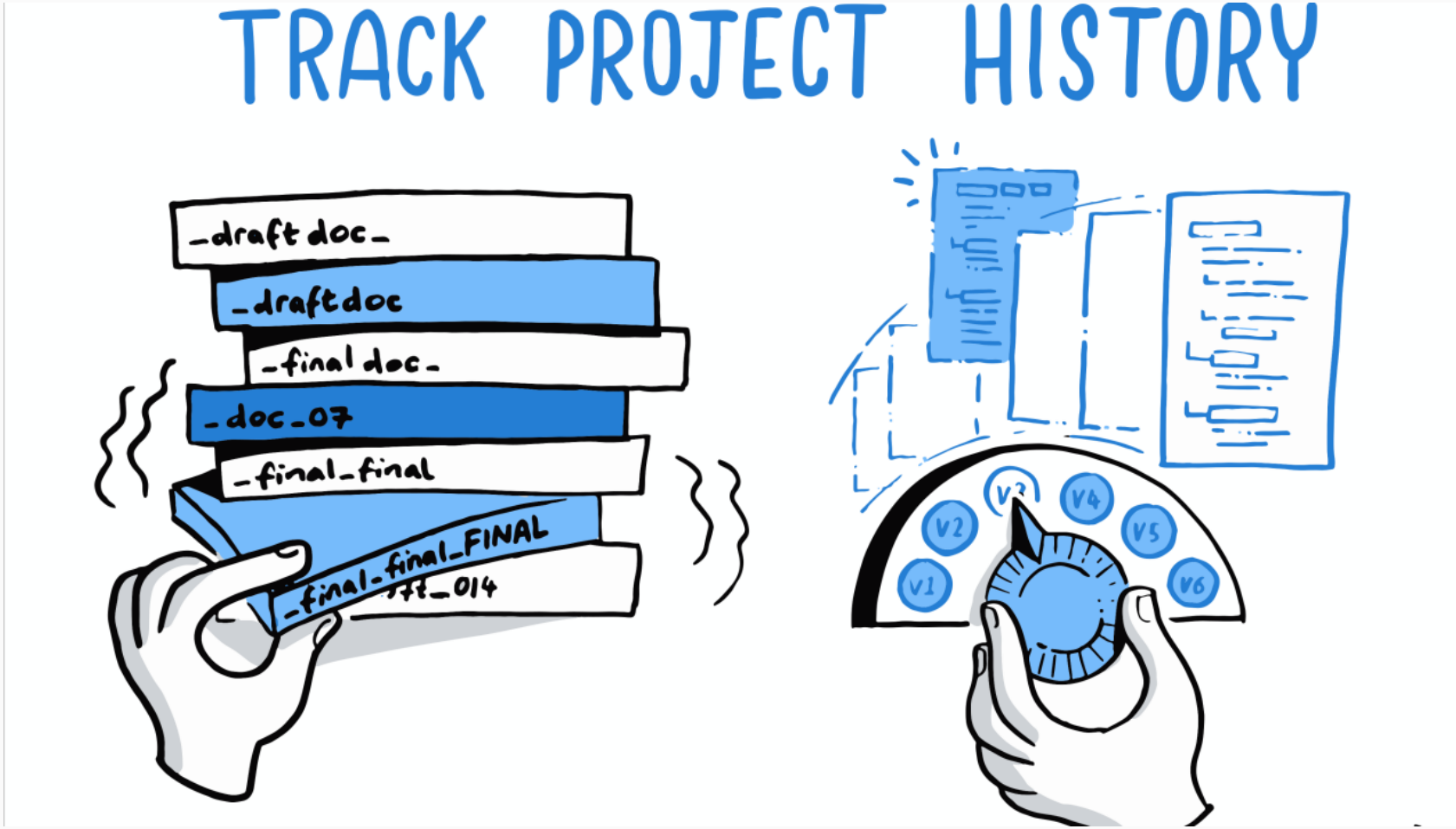

New session
What's new
New
Search
Projects
Beta
Artifacts
Scheduled
Customize
More
fec
U.S. contribution limits visual
fec
SNU Syllabi Slides
Choose environment
No folder
Recent
fec
SNU Syllabi Slides
Open folder...
Branch
main
Describe a task
This comp...
fec
Worktree
Fable 5.1 Extra
Auto
SK · Max
File
Edit
View
Help
What should we build?
Work in
This computer
Cloud
Remote
Choose project
This computer
Do anything
Approve for me
GPT-6.1 Sol Medium
3°C
맑음
Search
오전 8:25
2026-11-07

실습 1: Git 별도 설치

1. Windows: Git for Windows 설치 프로그램 실행
2. macOS: Apple ID로 Apple Downloads에서 Command Line Tools for Xcode 검색 후 설치; macOS 호환 버전 확인
3. 설치 후 AI 앱을 완전히 종료하고 다시 실행

폴더를 연 뒤 AI 앱에 **`git --version`** 실행을 요청해 버전 확인

버전 관리



출처: Opeyemi Oluwadare (2024), LinkedIn.

Git과 GitHub

Git 로고: Jason Long, CC BY 3.0.

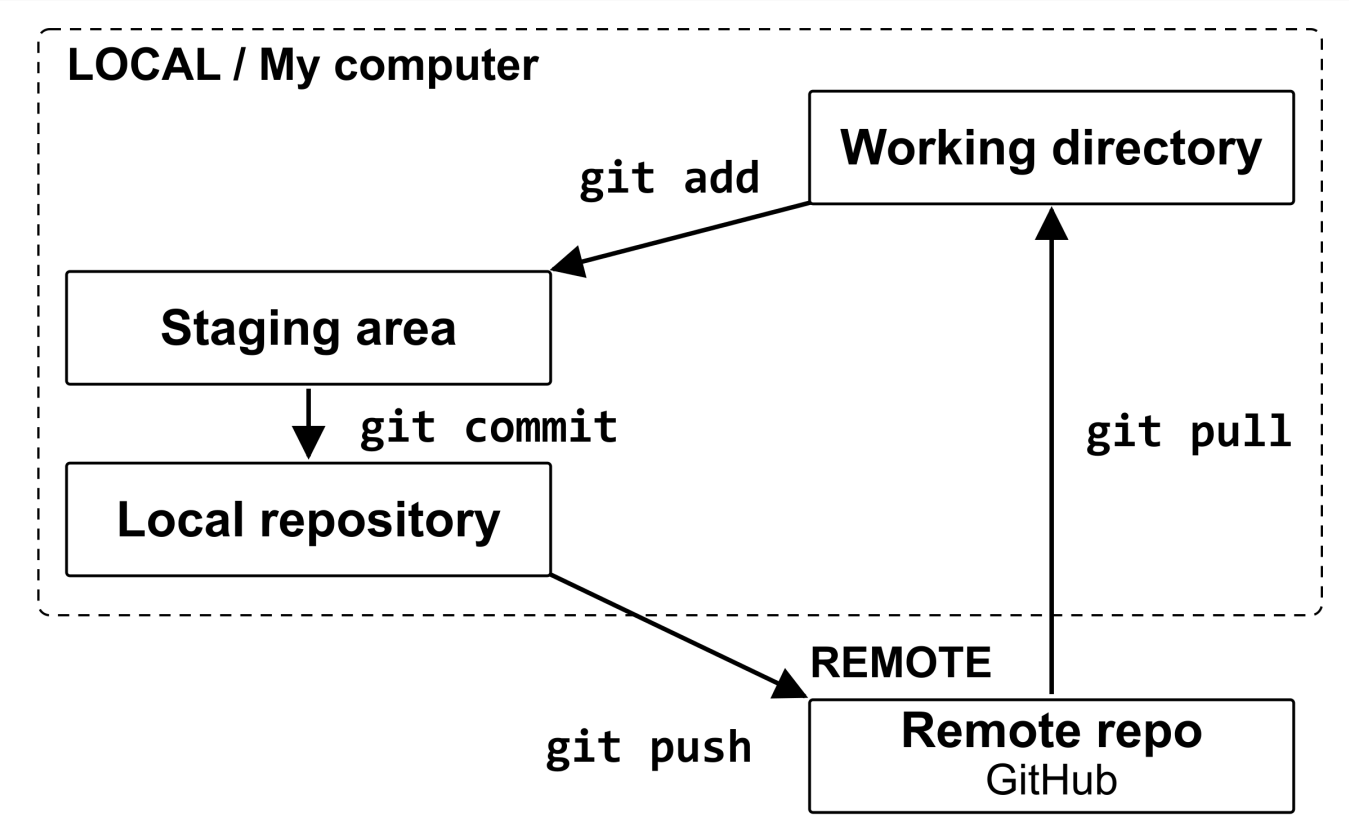
GitHub 로고: GitHub, Inc..

	Git	GitHub
정체	버전 관리 **프로그램**	Git 저장소를 올려두는 **웹 서비스**
위치	내 컴퓨터 (로컬)	클라우드 (원격)
하는 일	변경 이력을 스냅샷으로 기록	백업, 공유, 협업, 공개
없으면?	로컬 commit과 worktree 사용 불가	다른 원격 서비스나 별도 백업 가능

- Git: 인터넷 없이도 로컬 변경 이력 기록
- Git 작업은 AI 앱에 자연어로 요청

time machine

Git의 네 공간



참고: Git 공식 문서와 pull 문서

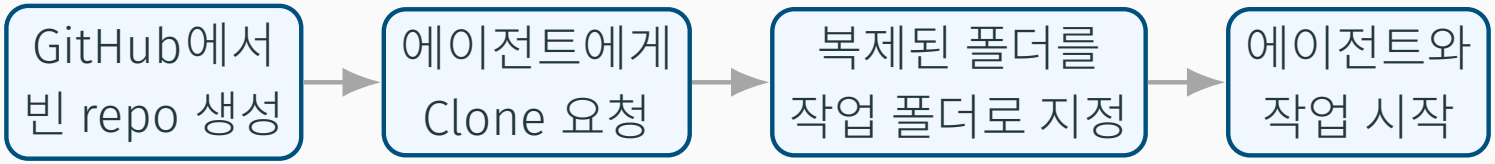
- Push는 로컬 커밋을 원격 저장소에 전송
- Pull은 원격 이력을 가져와 현재 브랜치에 통합

저장, 선택, 기록

단계	바뀌는 것	에이전트에 요청
파일 저장	작업 파일의 내용	파일에 저장
Add / staging	다음 커밋에 넣을 내용	커밋할 변경 선택
Commit	선택한 변경을 반영한 스냅샷	선택한 내용 커밋
Push	원격 저장소의 커밋 이력	확인 후 Push

- `hello.md` 저장만으로 커밋되거나 업로드되지는 않음
- add 후 다시 수정한 내용은 다시 add해야 포함됨
- 커밋 전에 새 파일과 수정한 줄을 확인하고 승인

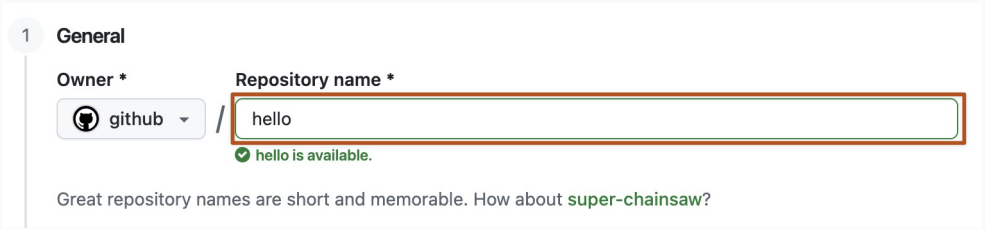
저장소와 작업 폴더

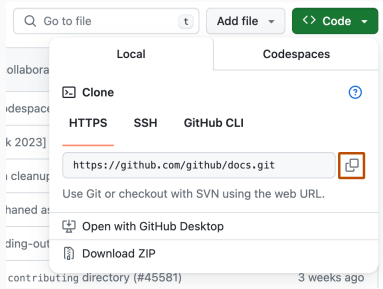


- 원격 저장소를 복제하면 로컬 폴더와 GitHub가 연결됨
- AI 앱: 파일 생성과 수정
- Git 작업도 에이전트에게 요청하고 결과 확인

실습 2: GitHub 저장소

1. github.com 로그인 → + → New repository
2. 이름: `kossda-lab`, Private 선택
3. Add a README file 선택 → Create repository
4. Code → HTTPS에서 저장소 주소 복사





출처: GitHub Docs: Creating a new repository, Cloning a repository, CC BY 4.0.

본인 계정의 비공개 저장소와 `README.md`가 보이는지 확인

실습 2: 폴더 복제

1. 파일 탐색기에서 빈 `kossda-lab` 폴더 만들기
2. AI 앱의 새 로컬 세션에서 이 폴더 선택
3. 아래 주소를 본인 저장소의 HTTPS 주소로 바꾸어 요청

현재 빈 폴더 자체를 저장소로 쓰도록 [내 저장소의 HTTPS 주소]를 하위 폴더 없이 복제해줘. 인증이 필요하면 브라우저 로그인으로 진행할 수 있도록 설정하고 로그인 완료를 기다려줘. 복제 후 현재 폴더의 Git 버전과 저장소 상태, 커밋 작성자 이름/이메일을 확인하고, 작성자 설정이 없으면 내게 물어본 뒤 이 저장소에 설정해줘.

작업 폴더에 `README.md`가 보이고, Git 버전과 저장소 상태가 확인되는지 확인

실습 2: 첫 파일

이 폴더에 hello.md 파일을 만들고, 그 안에 오늘 날짜와 “KOSSDA 실습 시작”이라는 문장을 적어줘. 그리고 방금 무엇을 했는지 한 문단으로 설명해줘.

- Claude는 **Manual**로 시작: 파일 변경 승인 과정을 관찰
- Codex는 **Ask for approval**로 시작: workspace 안 생성은 승인 없이 가능
- `hello.md`의 실제 생성과 내용을 파일 탐색기에서 직접 확인

`hello.md`를 직접 열어 생성 여부와 내용을 확인하고 요청과 일치하면 통과

실습 2: 변경안과 승인

hello.md의 기존 내용은 유지하고, 끝에 “진행 상태: 시작” 과 “다음 작업: 자료 확인” 두 줄을 추가하려고 해. 먼저 변경안만 보여주고 내 승인을 기다려. 다른 파일은 수정하지 마.

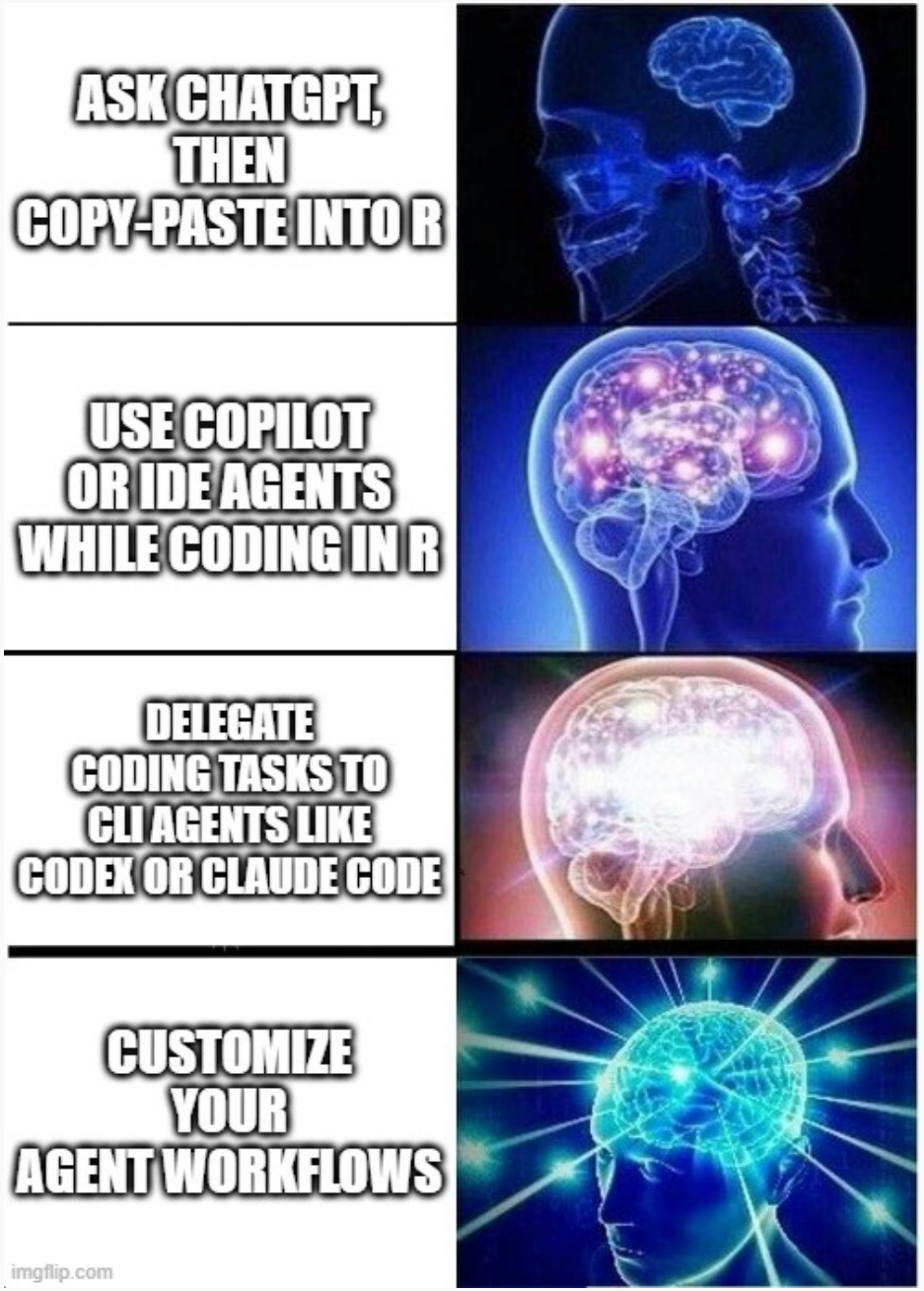
1. 변경안이 두 줄 추가인지 확인한 뒤 승인
2. 파일을 열어 기존 내용과 새 두 줄 확인
3. AI에게 새 파일을 포함한 변경 파일 목록 요청

`hello.md`에 요청한 내용만 추가되고 다른 파일은 바뀌지 않았으면 통과
새 파일은 전체 내용이 표시될 수 있음

정의와 작동원리

대화와 과업 위임

	대화 중심	에이전트형 과업 위임
상호작용	질문과 후속 대화	목표와 완료 조건 제시
반복	사용자 주도	**계획 → 행동 → 검토**
산출물	설명, 초안, 비교	파일, 실행 결과, 검토 가능한 결과물
도구 사용	검색과 분석도 가능	여러 도구와 단계를 자율적으로 연결
예시	ChatGPT Chat, 일반 채팅	ChatGPT Work, Codex, Claude Code

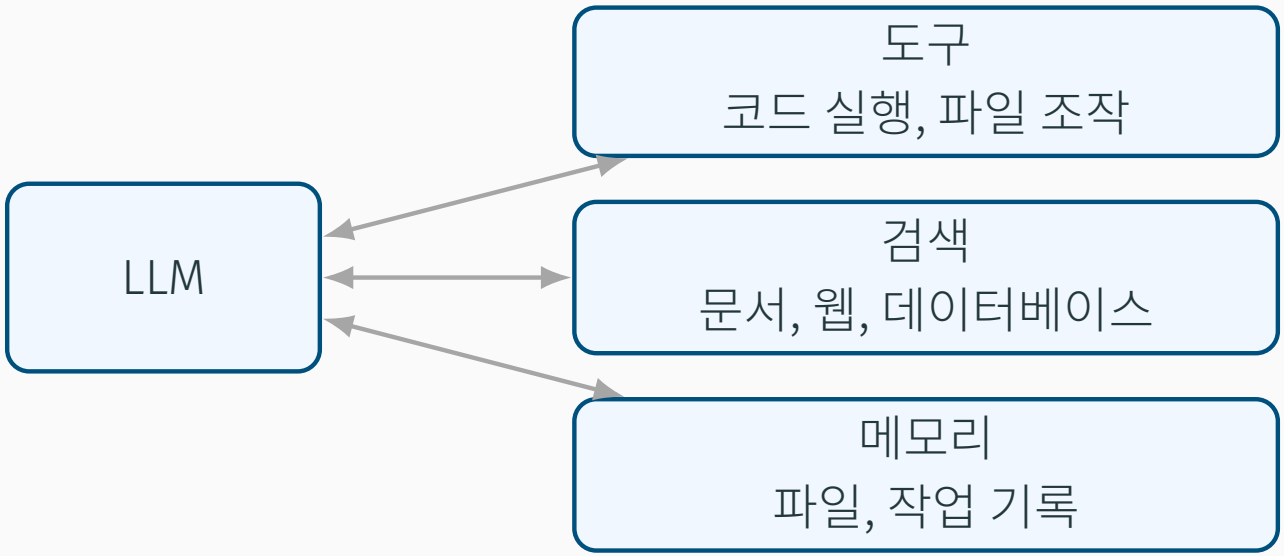

ASK CHATGPT,
THEN
COPY-PASTE INTO R
USE COPILOT
OR IDE AGENTS
WHILE CODING IN R
DELEGATE
CODING TASKS TO
CLI AGENTS LIKE
CODEX OR CLAUDE CODE
CUSTOMIZE
YOUR
AGENT WORKFLOWS
imgflip.com

Agentic AI의 핵심: Tool Use + 반복적인 자율 실행

- Tool use = LLM이 텍스트 생성을 넘어 외부 시스템과 직접 상호작용하는 것
- 대표적인 도구들:
  - **코드 실행**: R/Python 코드를 실행하고 결과를 읽음
  - **파일 조작**: 파일 읽기, 쓰기, 수정
  - **웹 검색**: 실시간 정보 검색
  - **터미널 명령**: shell 명령어 실행
  - **API 호출**: 외부 서비스 연결
- 결과를 다시 모델에게 피드백 → **반복적 자율 실행**

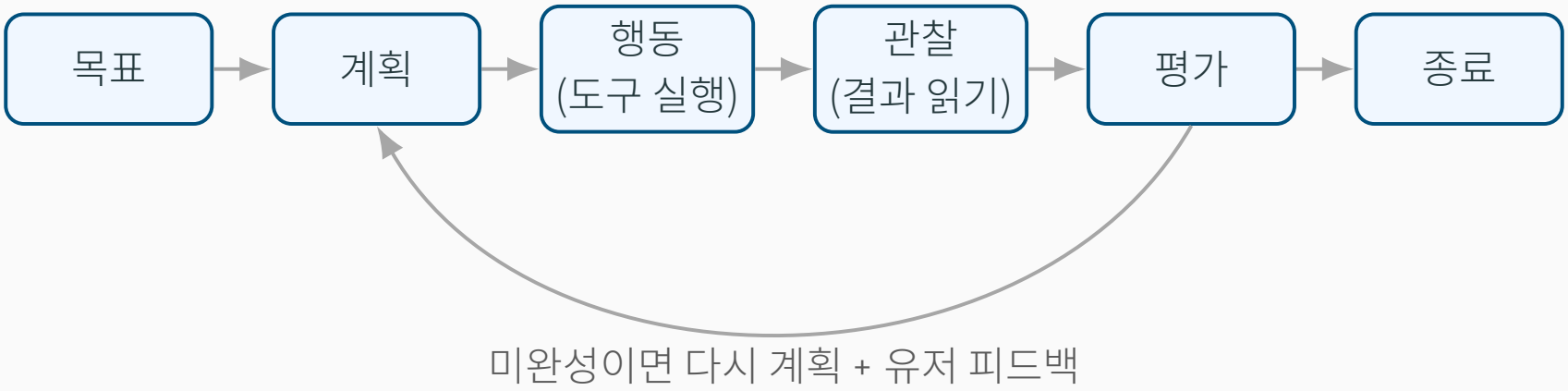
Augmented LLM

- 에이전트의 기본 단위 = augmented LLM
  LLM에 **도구(tools)**, **검색(retrieval)**, **메모리(memory)**를 연결



- Anthropic, “Building Effective Agents,” 2024

반복(Iteration)



반복과 중단

- R 실행 → 오류 확인 → 코드 수정 → 재실행
- 도구 실행 결과를 보고 다음 행동 선택
- 완료 조건, 비용/횟수 한도, 사람의 확인 지점에서 중단
- 반복이 길어지면 비용 증가; 첫 오류에서 반드시 회복되는 것은 아님
- 실행마다 경로와 결과가 달라질 수 있음

권한과 프라이버시

기준	통제 수단
무엇을 읽고 쓸 수 있나?	Filesystem scope, sandbox
무엇을 실행할 수 있나?	Tool allow/deny, approval
어디로 전송할 수 있나?	Network policy, connector 권한
외부 모델에 보내도 되나?	연구윤리, IRB, data governance

- Sandboxing $\neq$ data governance
- Sandbox: 컴퓨터에 대한 영향 범위
- Data governance: 외부 전송 가능 여부
- 최소 원칙: 필요한 폴더만, 사전 가림처리, 민감 자료 외부 전송 금지

CODEX: 권한 선택

UI 선택	자동 실행 범위	적합한 상황
Ask for approval	Workspace 안 읽기/편집/일반 명령	경계 밖 접근은 직접 검토
Approve for me	같은 sandbox; 추가 접근 요청은 자동 검토	일상 작업, 반복 검증
Full Access	모든 파일/명령/network 접근	격리된 환경, 신뢰된 프로젝트

- (저의 경우) 일상 기본값: **Approve for me**
- 두 모드의 차이 = **추가 접근 요청을 누가 검토하는가**
- 위험 작업은 사전 확인을 명시
  Workspace 안 파일 생성도 승인 없이 진행될 수 있음
- Full Access: 통제 최소, 책임 최대

Claude Code: 권한 선택

UI 선택	자동 실행 범위	적합한 상황
Manual	읽기 자동; 편집/명령은 승인	첫 실행, 세밀한 통제
Accept edits	파일 편집과 일반 filesystem 명령 자동으로 다른 terminal 명령은 승인	일반적 coding/문서 작업
Plan	읽기/분석/계획만	구조 파악, 변경 전 검토
Auto	승인이 필요한 행동을 classifier로 검토	검증된 반복 작업
Bypass Permissions	일반 승인 생략; 일부 보호는 유지	격리된 VM/container만

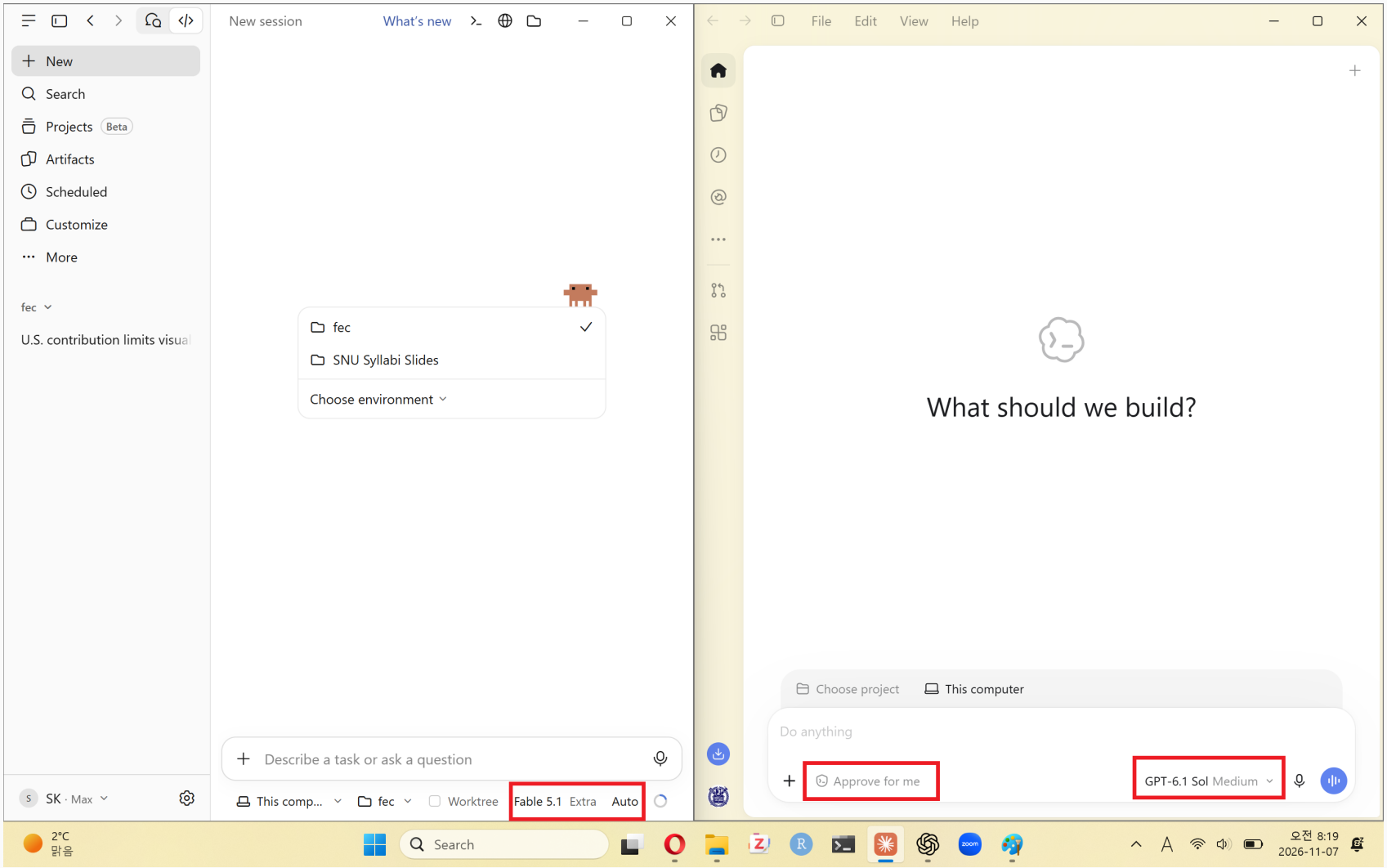
- 내부 mode: `default`, `acceptEdits`, `plan`, `auto`, `dontAsk`, `bypassPermissions`
- Auto $\neq$ Bypass Permissions (주의 필요)
- Auto의 지원 여부와 기본값은 모델, 앱 버전, 조직 정책에 따라 확인
- (저의 경우) Accept edits와 Auto를 왔다갔다 하는 편

실습 2: 권한 모드 바꿔보기

1. Claude: Manual에서 Accept edits로 바꾸고 `hello2.md` 생성
2. 파일 편집 승인 요청이 달라지는지 확인
3. Codex: Ask for approval과 Approve for me의 설명 비교
4. 추가 접근 요청의 검토자가 사용자 또는 자동 검토인지 확인
   (일부 모드는 계정/조직에 따라 없을 수 있음)
5. 실습이 끝나면 다시 원래 모드로 되돌리기

본인이 쓰는 앱에서 파일 편집과 추가 접근 요청을 어떻게 처리하는지 설명할 수 있으면 통과

실습 2: 권한 모드 바꿔보기



잠깐: Token이란?

- Token = 모델이 텍스트를 나누어 처리하는 단위
- 한 글자의 일부, 단어의 일부, 단어 전체 등이 하나의 token일 수 있음
- 영어는 대략 **1 token ≈ 영문 4글자**; 한국어의 비율은 다름
- 단어의 분할과 문서의 token 수는 **모델의 tokenizer**에 따라 달라짐
- 컨텍스트와 API 사용량은 token으로 측정
  구독 한도는 message/task, 시간 창, credit 등으로 표시되기도 함
- 정확한 비용/길이가 중요하면 모델의 tokenizer와 실제 usage log 확인

메모리: 컨텍스트 윈도우와 외부 기억

	단기 기억	장기 기억
위치	컨텍스트 윈도우 (대화 내부)	프로젝트 파일 (디스크)
수명	활성 context는 제한/압축 대화 기록은 저장/재개 가능	세션과 무관하게 유지
용량	유한 (모델별로 다름)	디스크는 크지만 필요할 때 다시 찾아 읽음
비용	길수록 token 비용 ↑	저장 + 검색 + 재독해 token/latency
예시	대화 내역, 읽은 파일	`CLAUDE.md`, `progress.md`, …

메모리: 컨텍스트 윈도우와 외부 기억

- 컨텍스트가 차면 **이전 대화를 요약하거나 줄임(compaction)**
- 대화에만 준 규칙은 요약 과정에서 빠질 수 있음
- Claude Code는 root `CLAUDE.md`를 압축 후 다시 읽음
  하위 규칙은 필요할 때 다시 로드
- **규칙** → **지시 파일**
  진행 상태 → `progress.md`
  (실제 준수 여부도 확인)

선택 실습: 새 세션 인계

progress.md에 현재 목표, 완료한 일, 지켜야 할 제약, 다음 할 일 한 가지를 기록해줘. 다른 파일은 수정하지 마.

새 세션에서 같은 폴더를 선택한 뒤 요청

progress.md와 hello.md를 읽고, 현재 상태와 다음 할 일만 보고해줘. 아직 실행하거나 파일을 수정하지 마.

새 세션의 보고를 두 파일과 대조하고, 추가 작업이 실행되지 않았으면 통과

RAG: 검색과 생성

- RAG: 관련 문서를 검색해 컨텍스트에 추가하고 답을 생성하는 방식
- 모델이 학습한 지식에 검색한 근거를 보강
- 질문 → 관련 대목 검색 → 프롬프트에 첨부 → 답 생성
- 검색한 문서와 답의 인용이 맞는지 확인 필요

RAG: 검색으로 근거를 보강하기

- 학습된 지식만으로는 최신 정보와 개별 주장의 출처를 보장하기 어려움
- 검색해 온 문서는 **출처를 추적**할 수 있고 최신 자료도 포함 가능
- 환각 위험을 줄여줌 (없애는 것 X)

예시

- **내가 수집한 PDF corpus**에 대한 질의응답
  - **관리형 RAG**: 논문 PDF를 NotebookLM, Claude Projects 같은 도구에 올리고, 모델이 그 문서를 근거로 답하게 하는 방식
  - “이 코퍼스에서 측정 오류를 다룬 논문과 각자의 정의를 모아줘”
  - **Zotero가 자동으로 RAG가 되는 게 아니고 Zotero를 대체하는 것도 아님**
    Zotero는 모으는 도구고 RAG는 모은 것을 참고하는 방법
- 주의: **garbage in, garbage out!** 검증 대체 X

잠깐: 프롬프트에 들어갈 것들

요소	예
목표	“핵심 변수의 분포를 요약”
입력	`sample.csv`, codebook, 참고 논문
제약/금지	“파일은 수정하지 마”, 패키지, 스타일, 경로
산출물	형식과 저장 위치: `summary.md`
검증	“행 수를 두 방법으로 교차 확인”
중단 조건	“불확실하면 멈추고 물어봐”

- 여섯 개 전부 매번 X; **산출물과 금지사항**은 거의 항상
- 긴 문장보다 **번호 목록**: 에이전트도 사람도 빠뜨리지 않음

실습 3: 자료 준비

1. 배포한 `fall-week1-student-kit.zip` 압축 해제
2. 아래 자료 세 개와 `course-notice.md`를 `kossda-lab`으로 복사; 기존 `README.md`는 유지
3. `source-codebook.md`에서 출처, 변수, 집계 기준 확인

- `sample.csv`: 직접 구성한 가상 설문 기록
- `source-codebook.md`: 자료 출처와 집계 기준
- `misleading-summary.md`: 검증할 모의 요약 초안
- 원자료 수정, 결측값 채우기, 중복 행 삭제 없이 진행

두 자료 파일과 코드북이 AI 앱의 작업 폴더에 보이면 통과. 실제 조사자료가 아니며 모집단으로 일반화하지 않음

실습 3: 구조화된 요청

source-codebook.md와 sample.csv를 읽어줘. (1) 행/열 수, 열 이름과 타입, 서로 다른 ID 수, 열별 결측값 수를 보고해줘. (2) policy_support의 비결측값 수, 합계, 평균을 계산하고 분모를 밝혀줘. (3) 중복 ID와 완전히 같은 행을 보고하고, 분포를 3문장으로 요약해줘. 모든 행을 유지하고 파일은 수정하지 마.

CSV를 직접 열어 행 수와 결측값을 대조하고, 계산기나 스프레드시트로 합계와 평균을 따로 확인했으면 통과

실습 3: 요청 비교

같은 앱과 모델, 같은 작업 폴더의 **새 세션**에서 요청

source-codebook.md를 참고해서 sample.csv 좀 봐줘.
모든 행을 유지하고 파일은 수정하지 마.

- 입력 파일과 수정 금지 조건은 동일하게 유지
- 응답 범위, 추측한 내용, 질문한 내용 비교
- 실제 파일이 그대로인지 확인

두 응답의 차이 또는 공통점을 실제 내용으로 한 가지 기록하면 통과

실습 3: 요약 초안 검증

misleading-summary.md의 세 주장을 sample.csv와 source-codebook.md에 대조해줘. 주장, 판정, 정정값, 근거가 되는 ID나 계산식과 분모를 표로 작성해 audit.md에 저장해줘. 원자료와 모의 초안은 수정하지 마.

1. CSV에서 해당 ID와 빈칸을 직접 찾기
2. 합계와 비결측값 수로 평균을 별도 계산
3. 행 수와 응답자 수를 구분했는지 확인

수치 두 개 이상과 해석 한 가지를 원자료에 대조하고
잘못된 부분을 정정했으면 통과

실습 3: 검증한 요약 저장

검증을 마친 세션에서 요청
새 세션이면 원자료와 코드북을 다시 읽도록 지정

sample.csv, source-codebook.md, audit.md를 읽고 원자료를 다시 계산해 summary.md를 작성해줘. 행 수와 서로 다른 ID 수, 결측값 수, policy_support의 합계/분모/평균, 가상 자료라는 해석 범위를 포함해줘. 출처는 코드북에 있는 내용만 쓰고, 오늘 날짜와 “AI 보조로 작성 후 수치 확인”을 적어줘. 원자료는 수정하지 마.

저장한 파일의 수치를 직접 확인한 계산값에 대조하고
출처와 해석 범위를 코드북에 대조했으면 통과

Workflow vs. Agent

	Workflow	Agent
구조	고정된 단계와 분기	상황에 따라 다음 행동 선택
LLM의 역할	정해진 노드에서 호출됨	루프 전체를 주도
예측가능성	높음: 같은 입력, 같은 절차	낮음: 실행마다 경로가 다름
적합한 작업	분류, 스코어링, routine한 과업	탐색, 디버깅, open task

- Workflow = 전체 과정의 입력/단계/출력/검증/종료 조건
- Agent = 그 과정 안에서 다음 행동을 상황에 맞춰 선택하는 실행자
- 응답 5,000건을 같은 프롬프트로 분류하는 스크립트 = **workflow**
- “이 merge 후 왜 행이 3배가 됐지?”를 파고드는 작업 = **agent**
- 단, workflow도 **절차만 고정**일 뿐 각 노드의 출력은 여전히 비결정적

Workflow Orchestration

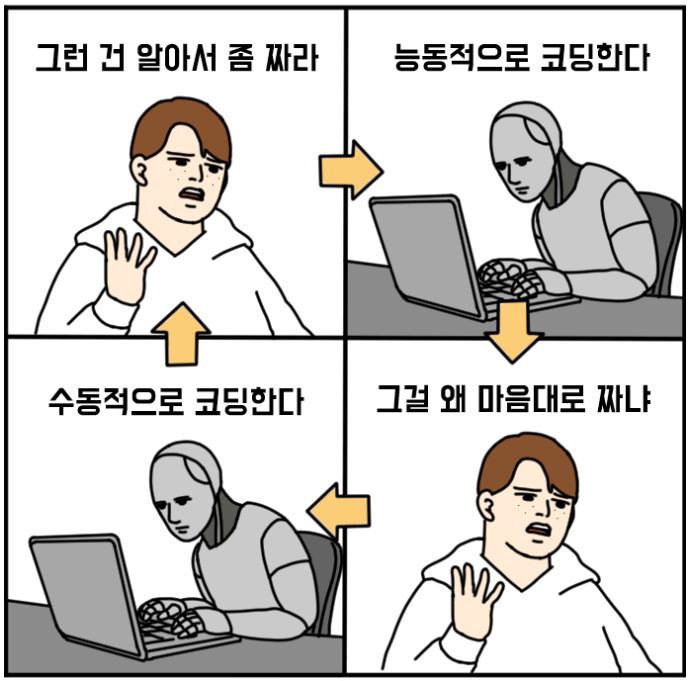
- **Orchestration**: 하나의 큰 과업을 작은 과업으로 나누고 각 worker에게 맡긴 뒤 결과를 다시 종합하는 방식
- 연구자의 역할 = 전체 과정을 설계/조정하는 orchestrator

단계	문헌 검토 예시
입력	연구 질문 + 검색 대상 문헌
분해	주제/방법별 검색 과업으로 나누기
위임	출처/주장/불확실성을 같은 형식으로 수집
종합	중복 제거, 주장 비교, 미해결 쟁점 정리
검증/종료	사람이 인용을 확인하고 연구자가 완료 여부 결정

경경수의 개발만화 29화: 지금개발

요즘은 다들 하나의 고민뿐이다
바로 AI를 어떻게 잘 써먹을까 하는 것이다

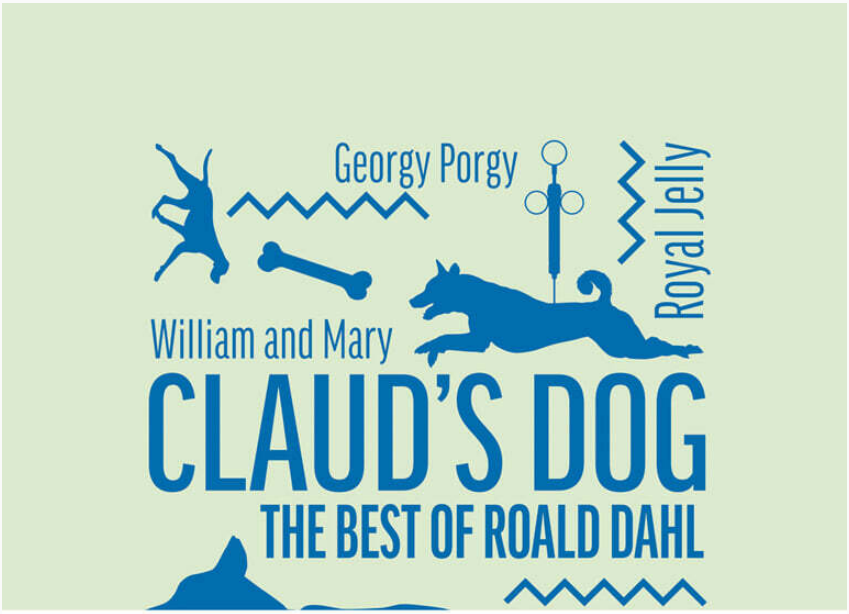
나같은 경우 수시로 말을 바꾸며
AI를 쉴 새 없이 부려먹거나



“AI를 어떻게 잡도리할 것인가”

마지막 실습에 앞서서...

- 일단 현실적인 이야기부터 하자면 효율적인 AI 잡도리를 할 수 있는 사람은 RAG니 에이전트니 개념을 정확히 아는 사람보다도, 내가 원하는 결과물을 AI 없이도 (시간이 주어진다면) 만들 수 있는 사람임
- 어떤 수준의 자동화에 대해서도 같은 원칙 적용
- Pre-AI 연구 경험이 있으면 날라다닐 수 있고 연구가 뭔지 잘 모르면 아무리 Fable이니 GPT Sol이니 가져다줘도 연구에 써먹을 수 없음
- 좋은 연구를 많이 참고하고 AI에게 사고외주하지 않는 시간을 충분히 확보하자



클로드의 개

로알드 달 소설

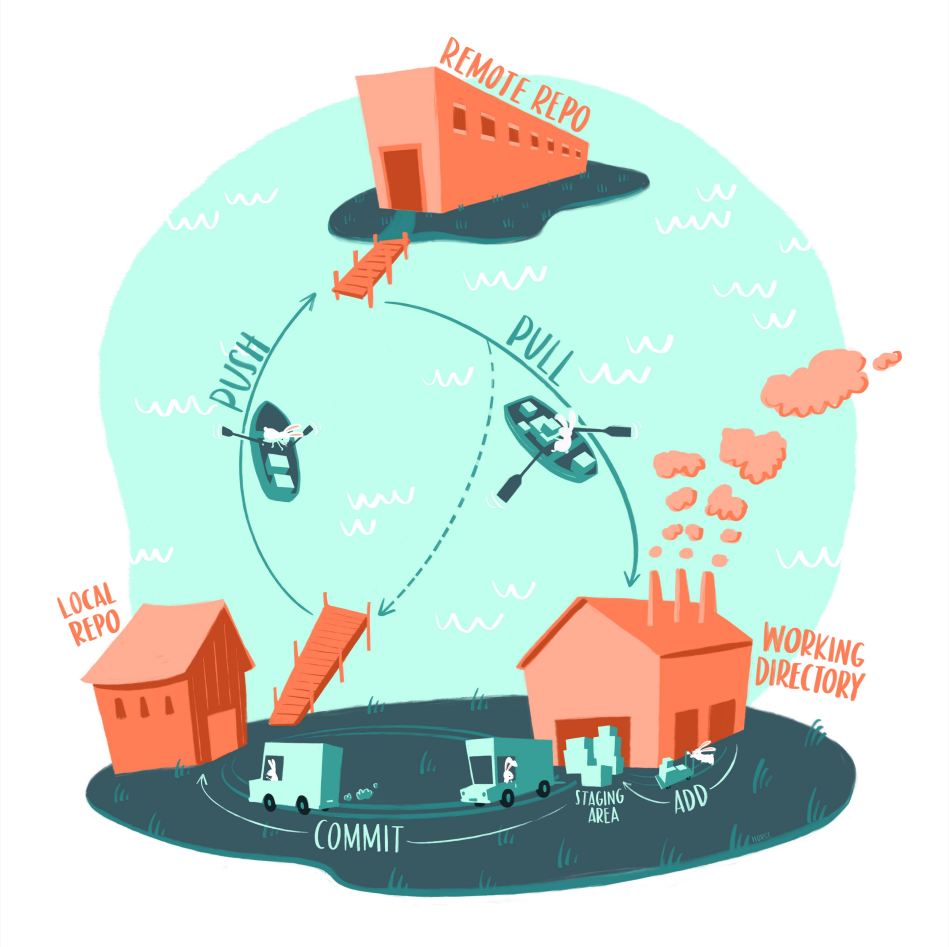
Roald Dahl

정영목 외 옮김

교유서가

버전 관리

로컬과 원격



Artwork by Allison Horst.

Commit과 Push

지금 한 일	내 컴퓨터	GitHub
파일 수정과 저장	작업 파일만 변경	그대로
Commit	새 스냅샷과 이력 기록	그대로
Push	로컬 커밋 유지	전송한 커밋 반영

- Commit은 오프라인에서도 가능; Push는 연결과 인증 필요
- Push는 커밋한 이력을 전송; 미커밋 수정은 업로드되지 않음
- 로컬 커밋 이력과 GitHub 웹의 커밋을 각각 확인
- Repository: 프로젝트의 파일과 변경 이력을 관리하는 저장소

변경 비교와 커밋 메시지

1. 새 파일을 포함한 변경 파일 목록 요청
2. 실제 파일과 추가, 수정, 삭제한 줄 확인: **diff**
3. 이번 작업에 해당하는 변경만 커밋하도록 지정
4. 바뀐 내용이 드러나는 한 줄 메시지 작성

- 예: `hello.md`에 진행 상태 추가
- 예: 자료 요약의 평균과 결측값 수 검증
- 서로 다른 작업은 나누어 커밋

Clone, Fetch, Pull

작업	하는 일	사용 시점
Clone	이력과 작업 파일을 처음 복제	이 컴퓨터에서 처음 시작
Fetch	원격 이력을 가져와 새 커밋 확인	변경 유무를 확인할 때
Pull	가져온 이력을 현재 브랜치에 통합	원격 변경을 반영할 때

- Clone 후에는 같은 로컬 폴더에서 계속 작업
- Fetch만으로 현재 작업 파일이 바뀌지는 않음
- 원격의 새 커밋을 확인한 뒤 현재 브랜치에 반영하도록 요청
- `origin`: 연결한 원격 저장소에 붙는 기본 이름

실습 4: 커밋과 업로드

새 파일을 포함해 변경 파일과 내용을 보여주고 내 승인을 기다려. 내가 승인한 변경만 stage하고 한 줄 메시지로 커밋한 뒤 GitHub에 Push해줘. 커밋 ID와 Push 결과를 알려줘.

1. 파일 목록과 diff를 검토한 뒤 커밋 승인
2. 보고된 커밋 ID와 실제 파일 내용 확인
3. GitHub 저장소의 커밋 목록에서 새 커밋과 변경 내용 확인

`hello.md`의 정상 내용이 커밋되고 GitHub에도 반영되었는지 확인

실습 4: 원격 변경 받기

로컬 작업을 커밋하고 Push한 뒤 시작

1. GitHub 웹에서 같은 브랜치의 `README.md` 열기 → 편집
2. 끝에 “원격에서 추가한 문장” 한 줄 추가 → **Commit changes**
3. 확인 창: **Commit directly to [현재 브랜치]** 선택 후 **Commit changes**

미커밋 변경이 있으면 멈추고 알려줘. 없으면 원격의 새 커밋을 가져와 보여주고 현재 브랜치에 반영한 뒤 README.md의 새 문장을 확인해줘. 이력이 갈라졌거나 충돌이 있으면 멈추고 알려줘.

로컬 `README.md`를 직접 열어 원격에서 추가한 문장이 보이는지 확인

브랜치와 병합

- Branch: 특정 커밋을 가리키며 작업에 따라 이어지는 이력의 이름
- `main`: 기본 브랜치 이름의 예; 별도 폴더를 뜻하지 않음
- 실험 브랜치에서 수정과 커밋 → 반영할 브랜치로 전환 → Merge
- Merge: 다른 브랜치의 변경을 현재 브랜치에 통합
- 충돌: Git이 자동으로 합칠 내용을 결정하지 못한 상태
  최종 내용을 확인하고 해결

변경 취소와 복구

상태	작업	에이전트에 요청
아직 커밋하지 않음	미커밋 수정 취소	이 파일만 복구
이미 커밋함	상쇄하는 새 커밋	해당 커밋을 Revert

- 복구 전에는 대상 파일과 바뀔 내용을 확인
- Revert는 기존 커밋을 지우지 않음; 원격 반영에는 다시 Push
- 복구할 내용이 이력에 기록되어 있는지 먼저 확인
- 다음 실습: 정상 상태를 커밋한 `hello.md` 하나만 수정하고 복구

실습 4: 한 파일 복구

`hello.md`의 정상 상태가 커밋되어 있는지 먼저 확인

> hello.md의 내용만 전부 지우고 “실수” 라고 적어줘. 다른 파일은 수정하지 마.

> hello.md의 미커밋 수정만 마지막 커밋의 내용으로 되돌려줘. 사용할 명령을 먼저 보여주고 내 승인을 기다려. 다른 파일은 수정하지 마.

1. 복구 대상이 `hello.md` 하나인지 확인한 뒤 승인
2. 파일을 직접 열어 정상 내용과 대조

`hello.md`가 정상 내용으로 돌아오고 다른 파일이 그대로인지 확인

접속과 Git 문제

- 앱 로그인 실패: 지원 계정과 기본 브라우저 확인, 기관 VPN 해제 후 재시도
- 한글 폴더 경로에서 오류: 영문 경로의 폴더로 이동
- 비공개 저장소 복제 실패: 저장소 주소와 Git 인증 확인
- 커밋 실패: Git의 작성자 이름과 이메일 확인
- AI 앱의 Git 기능 오류: Git 별도 설치 후 앱 재시작
- GitHub 웹 로그인과 로컬 Git 인증은 별개
- 사용량 한도 도달: 표시된 재개 시점 확인

개인 프로젝트

개인 프로젝트 초안

작은 연구 과업 하나의 계획을 project-brief.md로 정리하려고 해. 목표, 사용할 수 있는 자료, 산출물 한 가지, 독립적으로 확인할 방법, 다음 행동을 먼저 질문해줘. 답을 받은 뒤 초안만 저장하고 실제 작업은 시작하지 마.

- 오늘 강사와 범위와 실행 가능성 협의
- 자료 접근과 처리 권한, 완성 기준 확인
- 최종 산출물: **4주차 시작 전 제출**

다섯 항목을 담은 초안이 있고 강사와 협의할 과업이 정해졌으면 통과

다음 주 준비

- 개인 프로젝트 초안을 강사와 협의하고 범위 확정
- `project-brief.md`에 다음 작업과 확인 방법 기록
- 선택: 작은 과업 하나를 더 실행하고 결과와 문제 한 가지씩 기록
- Week 2: 모델/effort, 지시 파일, rules/skills/hooks/MCP, 비용과 한도

터미널 참고

참고: 터미널

- 마우스 대신 **명령어를 타이핑**해서 컴퓨터에 일을 시키는 창
- macOS: `Terminal.app` vs. Windows: `PowerShell`
- 다음 슬라이드들의 회색 상자 = **터미널에 한 줄씩 붙여넣고** Enter
- `#`으로 시작하는 줄은 설명(주석)이라 실행되지 않음

오늘의 기본 경로에서는 직접 명령을 입력하지 않아도 됩니다

참고: Codex CLI 설치

npm 또는 Homebrew로 설치

```
# npm
npm install -g @openai/codex

# macOS (Homebrew)
brew install --cask codex
```

- CLI 설치 후 프로젝트 폴더에서 `codex` 실행; 데스크탑 앱과는 별도 설치
- ChatGPT 계정(Free/Go/Plus/Pro/Business/Edu/Enterprise)
  또는 OpenAI API 키로 인증
- **그런데 그냥 ChatGPT Desktop App을 쓰는 걸 추천**

참고: Claude Code CLI

Native 설치 (권장): Node.js 없이 설치

```
# macOS, Linux, WSL
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell
irm https://claude.ai/install.ps1 | iex

# Windows CMD
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del install.cmd
```

또는 npm (Node.js 22 이상 필요): `npm install -g @anthropic-ai/claude-code`

- CLI 설치 후 프로젝트 폴더에서 `claude` 실행; 데스크탑 앱에는 Code가 포함됨
- Pro/Max/Ultra/Team/Enterprise **구독** 또는 Console(API) 계정
  Free 플랜은 불가
- **데스크탑 앱 쓰세요 웬만하면**

Desktop vs. CLI: Claude Code

	Desktop App	CLI
세션	사이드바, 병렬 세션, 시각적 검토	터미널별 세션, `--resume`
입력	파일 첨부, PDF/이미지	파일 경로, 명령줄 중심
자동화	예약 작업, Computer Use	`--print`, Agent SDK, CI
강점	여러 작업 관리, diff 검토	스크립팅, SSH, 터미널 작업

- **같은 엔진 + 같은 설정**: `CLAUDE.md`, settings, MCP, hooks, skills
- **별도 세션 기록**. CLI의 `/desktop`으로 세션 이동 가능

Desktop vs. CLI: Codex

	ChatGPT Desktop	Codex CLI
환경	Local / worktree / cloud	로컬 프로젝트 폴더
세션	여러 task를 한 창에서 관리	터미널 중심 대화
산출물	문서/표/그림/브라우저 검토	코드/파일/명령 실행
자동화	Scheduled task, plugins	`codex exec`, scripts, CI
강점	병렬 작업 + 시각적 검토	빠른 로컬 작업 + 다른 터미널 코드와 조합 가능

- 공통 기반: `AGENTS.md` + skills + **프로젝트 파일**
- Desktop = command center vs. CLI = terminal-first local agent
- 터미널에서 왔다갔다 가능 e.g., `codex "refactor current codebase"`
- 이 수업 듣는 분들 중에 터미널에서 주로 작업하는 분은 거의 없을 걸로 아뢰오. **데스크탑 앱 씁시다**

선택 실습: Git 명령 확인

1. 터미널 열기: macOS는 **`Terminal.app`**, Windows는 **`PowerShell`**
2. 아래 명령을 입력하고 Enter

```
git --version
# 출력 예: git version 2.55.0.windows.3
```

`git version 2.x.x` 형태의 버전이 출력되면 통과
"명령을 찾을 수 없음" 오류가 나면? PATH에 Git이 없음: git-scm.com에서 설치 후 터미널을 새로 열어 재확인

선택 실습: CLI로 복제

1. 아래 **두 방법 중 하나만** 실행; clone 주소는 본인 저장소로 교체

```
cd ~/Documents
git clone https://github.com/USERNAME/kossda-lab.git

# GitHub 계정이 없는 경우에만: 로컬 저장소 생성
mkdir ~/Documents/kossda-lab
cd ~/Documents/kossda-lab
git init
```

2. 처음이면 브라우저 로그인 창(Git Credential Manager)이 뜰 수 있음
   본인 계정으로 승인
3. 데스크탑 앱에서 새 세션을 열고 작업 폴더로 `kossda-lab` 선택
4. 로컬 대안은 "이 폴더에 실습용 README.md를 만들어줘"라고 요청

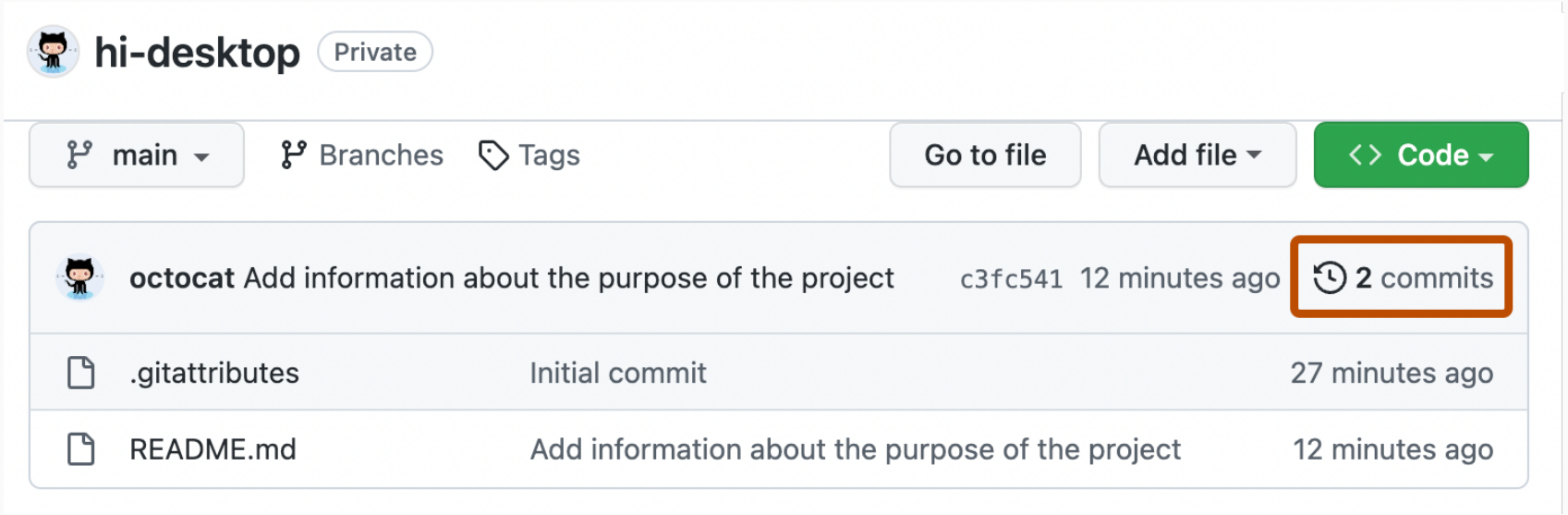
앱에 폴더 경로(`kossda-lab`)가 표시되고
폴더 안에 `README.md`가 보이면 통과

선택 실습: 에이전트 Git 조작

Git 설치, 저장소 상태, 커밋 작성자 설정을 먼저 확인해줘. 지금까지 만든 실습 파일 중 커밋할 파일 목록과 변경 내용을 보여줘. 내가 확인한 파일만 커밋해줘. GitHub가 연결되어 있으면 푸시도 해줘. 커밋 메시지는 한 줄로, 무엇을 추가했는지 적어줘. 실행 전에 명령을 보여주고 내 승인을 기다려.

1. 에이전트가 보여주는 명령(`git add`, `git commit`, `git push`)을 읽고 승인
2. 브라우저에서 GitHub 저장소 새로고침:
   `hello.md`, `summary.md`가 보이는지 확인
3. 커밋 메시지를 클릭해 무엇이 바뀌었는지(diff) 눈으로 확인

선택 실습: 에이전트 Git 조작



출처: GitHub Docs, CC BY 4.0.

GitHub 페이지에서 파일과 커밋 메시지가 보이면 통과
(급한대로) GitHub 계정이 없는 분은 푸시 없이 커밋까지만!

선택 실습: 명령으로 복구

hello.md의 내용을 전부 지우고 “실수”라고만 적어줘.

방금 hello.md를 망쳤어. 마지막 커밋 상태로 되돌려줘.
되돌리기 전에 어떤 명령을 쓸지 보여주고, 다른 파일은 건드리지 마.

- 되돌리는 명령이 `hello.md` 하나만 대상으로 하는지 확인
  (`git restore --source=HEAD -- hello.md`)
- **`git reset --hard`가 나오면 승인하지 말 것**
  커밋 안 한 다른 변경까지 전부 사라짐
- 커밋하지 않은 작업은 이 안전망 밖 → 자주 커밋

`hello.md`가 원래 내용으로 돌아왔고 다른 파일은 그대로면 통과
먼저 정상 상태를 커밋한 파일로만 진행